

Contact Manager

ATTSW Project Report

Student: Mohamed Jama Farah

Professor: Lorenzo Bettini

University: University of Florence

Course: ATTSW — A.Y. 2025/2026

Exam date: July 20, 2026 — 09:30 — Aula 005 C.Didat.Morgagni

GitHub: github.com/mohamedjama-farah/Contact-Manager

1. Introduction

This report covers the design, implementation, and testing of the Contact Manager application, built as the final project for the ATTSW course at the University of Florence. The app lets users add, view, and delete contacts, with everything persisted in MongoDB.

I built the project with Test-Driven Development from start to finish: no class or method went in before a failing test asked for it. Along the way it touches every part of the quality toolchain covered in the course — unit tests, integration tests, end-to-end tests, coverage enforcement, mutation testing, static analysis, and continuous integration.

1.1 Project Goals

- Implement the Contact Manager application using the MVP architectural pattern.
- Write all production code through the TDD cycle: Red → Green → Refactor.
- Reach and enforce 100% line and branch coverage with JaCoCo.
- Reach a 100% mutation score with PIT (0 survived mutations).
- Wire up SonarCloud for static analysis (0 issues across every quality dimension).
- Wire up Coveralls for coverage tracking and the badge display.
- Automate the whole quality pipeline with GitHub Actions.

2. Architecture — MVP Pattern

The application follows the Model-View-Presenter pattern, which keeps each component responsible for one thing and testable on its own. The controller only knows about interfaces — it has no dependency on Swing or MongoDB — so it's fully testable with Mockito fakes.

ContactController depends only on the ContactRepository and ContactView interfaces, which means it can be tested without starting a database or a GUI: you just inject Mockito mocks. The real implementations, MongoContactRepository and ContactFrame, only get wired together in ContactManagerApp when the application starts.

2.1 Key Design Decisions

- Interfaces for testability: ContactRepository and ContactView are interfaces rather than concrete classes, which is what makes it possible to unit test the controller in isolation.
- ContactManagerApp is left out of coverage: its main() method just wires components together and isn't meaningful to unit test, so it's excluded from JaCoCo.
- setController() on ContactFrame: the frame needs a reference to the controller for its button actions, but the controller also needs the frame. That circular dependency gets resolved by setting the controller after both objects exist.

Layer	Class / Interface	Role
Model	Contact.java	Stores name, phone, email, id
Model	ContactGroup.java	Groups contacts by category
Repository	ContactRepository	Interface: save(), findAll(), delete()
Repository	MongoContactRepository	MongoDB implementation of the repository
Controller	ContactController	Business logic — bridges repository and view
View	ContactView	Interface: showAllContacts(), showError(), etc.
View	ContactFrame	Swing implementation of the view
App	ContactManagerApp	main() — wires all components together

3. Test-Driven Development

Every production class in this project came out of the TDD cycle. There's no production code without a test that demanded it first.

3.1 Example: `ContactController.addContact()`

Here's what the TDD steps actually looked like for `addContact()`:

Step 1 — RED: write the test first

```
@Test

public void testAddContactShouldSaveToRepositoryAndUpdateView() {

    controller.addContact("Mohamed", "0039123456789", "m@example.com");

    verify(repository).save(any(Contact.class));

    verify(view).contactAdded(any(Contact.class));

}
```

Running it fails, as it should: `addContact()` doesn't exist yet.

Step 2 — GREEN: write the minimum code

```
public void addContact(String name, String phone, String email) {

    Contact contact = new Contact(name, phone, email);

    repository.save(contact);

    view.contactAdded(contact);

}
```

Running it now passes — the test is satisfied with the minimum needed.

Step 3 — REFACTOR

Nothing to clean up here; the code was already simple enough.

Phase	Colour	What happens
1. Write test	Red	Write a test for functionality that doesn't exist yet, and run it — it has
2. Write code	Green	Write the minimum code needed to make the test pass. Nothing
3. Refactor	Blue	Clean up the code without changing behaviour. Tests still have to pass.

4. Test Strategy

The project runs three levels of testing, each with a different scope, speed, and set of dependencies.

4.1 Unit Tests

ContactControllerTest checks the business logic on its own: Mockito supplies fake implementations of ContactRepository and ContactView, so the controller runs without touching a database or GUI. Three test methods cover allContacts(), addContact(), and deleteContact().

ContactFrameTest exercises the Swing GUI with AssertJ-Swing. It builds a real ContactFrame, wraps it in a FrameFixture that simulates user interaction, and mocks the controller with Mockito. CI runs it against a virtual X11 screen (Xvfb). Five tests cover showing contacts, showing error dialogs, clicking Add, clicking Delete with a selection, and clicking Delete with nothing selected.

4.2 Integration Tests

MongoContactRepositoryIT tests the MongoDB repository against a real database. Testcontainers starts a MongoDB 6.0 Docker container via @ClassRule, and the same container is shared across every test in the class. Three tests cover save-and-findAll, delete, and close(). These run during the Failsafe integration-test phase.

4.3 End-to-End Tests

ContactManagerE2EIT drives the whole stack: a real MongoDB container comes up, a real ContactFrame is shown, and a real ContactController wires them together. The test types a contact, clicks Add, checks it shows up both in the list and in the database, then selects it, clicks Delete, and confirms it's gone.

Level	Files	Tools	Dependencies
Unit Tests	ContactControllerTest, ContactFrameTest, ContactTest, ContactGroupTest	JUnit 4 + Mockito + AssertJ-Swing	None (fakes only), virtual screen
Integration	MongoContactRepositoryIT	JUnit 4, Testcontainers	Real MongoDB in Docker
End-to-End	ContactManagerE2EIT	AssertJ-Swing, Testcontainers	Full stack: GUI + DB in Docker

5. Build Automation — Maven

Maven handles compilation, dependency resolution, and test execution through its standard lifecycle. pom.xml sets up every plugin and dependency the project needs.

5.1 Maven Lifecycle for this Project

Phase	Plugin	What runs
-------	--------	-----------

compile	compiler	Compiles src/main/java (8 source files)
test-compile	compiler	Compiles src/test/java and src/it/java (6 test files)
test	Surefire	Runs *Test.java — unit tests only (excludes *IT.java)
integration-test	Failsafe	Starts Docker; runs *IT.java integration and E2E tests
verify	JaCoCo check	Fails the build if line or branch coverage drops below 100%
sonar:sonar	SonarCloud	Sends analysis to sonarcloud.io
coveralls:report	Coveralls	Sends the coverage report to coveralls.io

5.2 JaCoCo — Coverage Enforcement

JaCoCo instruments the bytecode to track which lines and branches actually run during tests. pom.xml sets up three executions:

- prepare-agent: injects the JaCoCo agent into the JVM before tests run, filling the <argLine> placeholder in the Surefire and Failsafe config.
- report: once tests finish, generates the XML and HTML coverage report under target/site/jacoco/.
- check: reads the report and fails the build if line or branch coverage falls below 1.0 (100%). ContactManagerApp.class is excluded since it's just main() and isn't unit-testable.

5.3 PIT — Mutation Testing

PIT checks whether the tests are actually meaningful and not just padding out coverage numbers. It mutates the production bytecode, reruns the tests against each mutant, and checks whether anything fails. A failing test kills the mutant, which is what you want; if every test still passes, the mutant survives, meaning the tests are too weak to catch that change.

Metric	Result
Mutations generated	5
Mutations killed	5 (100%)
Mutations survived	0
Test strength	100%
Line coverage	14/14 (100%)

6. Continuous Integration — GitHub Actions

Every push, on any branch, kicks off the workflow defined in .github/workflows/build.yml. It runs on ubuntu-latest and runs the full quality pipeline in a single command.

6.1 Workflow Steps

Step	Details
checkout@v4	Clones the repository with full git history (fetch-depth: 0), which SonarCloud needs for accurate analysis.
setup-java@v4	Installs Java 17 (Temurin distribution).
xvfb-run mvn verify	Creates a virtual X11 screen so AssertJ-Swing can open real windows during the GUI tests, then runs the full Maven lifecycle.
sonar:sonar	Sends source, bytecode, and the JaCoCo XML report to SonarCloud, using SONAR_TOKEN from GitHub Secrets.
coveralls:report	Sends the JaCoCo XML to Coveralls, using COVERALLS_REPO_TOKEN from GitHub Secrets.

6.2 Secrets Configuration

Secret name	Purpose
SONAR_TOKEN	Authenticates Maven with SonarCloud
COVERALLS_REPO_TOKEN	Authenticates Maven with Coveralls
GITHUB_TOKEN	Provided automatically by GitHub Actions

6.3 Docker and Testcontainers in CI

Testcontainers spins up a real MongoDB 6.0 Docker container during the integration-test phase. GitHub Actions' ubuntu-latest runners already have Docker installed, so there's no extra setup. The `@ClassRule` annotation makes sure the container starts once and gets shared across every test in `MongoContactRepositoryIT` and `ContactManagerE2EIT`.

7. Quality Results

These numbers come straight from the live CI pipeline and the quality dashboards.

7.1 Test Count

Metric	Result	Tool
Line coverage	100%	JaCoCo / SonarCloud / Coveralls
Branch coverage	100%	JaCoCo

Mutation score	100%	PIT
Mutations survived	0	PIT
Security issues	0	SonarCloud
Reliability issues	0	SonarCloud
Maintainability issues	0	SonarCloud
Hotspots reviewed	100%	SonarCloud
Duplications	0.0%	SonarCloud
CI builds passing	Yes	GitHub Actions

Test class	Type	Count	Runner
ContactTest	Unit	1	Surefire
ContactGroupTest	Unit	2	Surefire
ContactControllerTest	Unit	3	Surefire
ContactFrameTest	Unit (GUI)	5	Surefire
MongoContactRepositoryIT	Integration	3	Failsafe
ContactManagerE2EIT	End-to-End	1	Failsafe
TOTAL		15	

8. Key Challenges and Solutions

8.1 Missing branch coverage (99.2% → 100%)

JaCoCo reported 99.2% coverage. Digging into jacoco.xml showed the false branch of `if (index != -1)` inside the delete-button listener in ContactFrame never ran. The fix was adding `testDeleteButtonDoesNothingWhenNoItemSelected()`, which clicks Delete with nothing selected in the list and exercises that missing branch.

8.2 SonarCloud 'Add at least one assertion' warning

SonarCloud flagged `testShowErrorDisplaysDialog()` because it only used the AssertJ-Swing API without an explicit AssertJ assertion. The fix was storing the `JOptionPaneFixture` in a variable and adding `assertThat(optionPane).isNotNull()` before the fluent chain.

8.3 Coveralls token not picked up automatically

The Coveralls Maven plugin wasn't reading the COVERALLS_REPO_TOKEN environment variable on its own. The fix was passing it explicitly on the command line: coveralls:report -DrepoToken=\$COVERALLS_REPO_TOKEN

8.4 GUI tests need a virtual display in CI

AssertJ-Swing and ContactManagerE2EIT open real Swing windows, and GitHub Actions runners have no physical display. The fix was prefixing the Maven command with xvfb-run --auto-servernum, which creates a virtual X11 framebuffer display.

8.5 PIT 'No mutations found' on first run

PIT needs compiled .class files in target/classes before it can generate mutations. Running the plugin before compiling threw this error. The fix was running mvn test-compile first to populate target/classes, then running PIT.

9. Source Code Structure

```
contact-manager/  
  
|-- pom.xml  
  
|-- .github/workflows/build.yml  
  
|-- README.md  
  
|-- src/  
  
    |-- main/java/com/example/contactmanager/  
  
        |-- app/ContactManagerApp.java  
  
        |-- controller/ContactController.java  
  
        |-- model/Contact.java  
  
        |-- model/ContactGroup.java  
  
        |-- repository/ContactRepository.java  
  
        |-- repository/MongoContactRepository.java  
  
        |-- view/ContactView.java  
  
        |-- view/swing/ContactFrame.java  
  
    |-- test/java/com/example/contactmanager/  
  
        |-- controller/ContactControllerTest.java
```



```
| |-- model/ContactTest.java

| |-- model/ContactGroupTest.java

| |-- view/swing/ContactFrameTest.java

|-- it/java/com/example/contactmanager/

|-- repository/MongoContactRepositoryIT.java

|-- e2e/ContactManagerE2EIT.java
```

9.1 GitHub Repository

<https://github.com/mohamedjama-farah/Contact-Manager>

The repository's git history shows the TDD progression — each commit maps to a Red, Green, or Refactor step.

10. Conclusion

The Contact Manager project pulls together the software quality techniques covered in ATTSW. Sticking to TDD meant every line of code existed because a test demanded it, and the mutation testing results — 0 survived mutants — back up that the test suite is catching real defects, not just padding coverage numbers. The CI pipeline keeps those standards in place on every future commit.

Objective	Achieved
TDD applied to all production classes	Yes
MVP architectural pattern	Yes
Unit, integration, and end-to-end test coverage	Yes (15 tests)
100% line and branch coverage (JaCoCo)	Yes
100% mutation score (PIT, 0 survived)	Yes
0 SonarCloud issues (Security/Reliability/Maintain.)	Yes
Continuous Integration with GitHub Actions	Yes
Coveralls integration with 100% badge	Yes